Code review Ali Raza

Type	
Created by	Ⓐ Armando
Created time	April 9, 2026 11:32 PM
Last edited by	Ⓐ Armando
Last edited time	April 9, 2026 11:32 PM
Authors	🇨🇱 Armando

Technical Code Review: Ali Raza — Geo-Sdk (Bounty Uploader)
- Architecture Overview
- Issues Found
  - Critical
  - Moderate
  - Minor
- What's Done Well
- Summary

Technical Code Review: Ali Raza — Geo-Sdk (Bounty Uploader)

Repository: Raza-Khan92/Geo-Sdk
SDK version: `@geoprotocol/geo-sdk@^0.7.0` **Runtime:** Node.js + `tsx` **Datasets:** Block Explorers (65 records), Networks (39 records)

Architecture Overview

This repo takes a fundamentally different approach from other bounty submissions. Instead of hardcoding entity creation per bounty, it implements a **config-driven generic uploader**:

```
bounties/<name>/
  ├── <name>-config.json   ← schema definition (entity type,
```

```
fields, relations)
  └── <name>-data.json     ← array of records matching config
keys
```

The SDK has 4 source files:

- `sdk/upload.ts` — CLI entry point, detects config vs data files automatically
- `sdk/core/importer.ts` — Three-phase schema resolution + record import with dedup
- `sdk/core/graph-client.ts` — GraphQL client, search, and publish (personal + DAO spaces)
- `sdk/core/types.ts` — TypeScript interfaces for config, fields, resolved schema
- `sdk/core/constants.ts` — Well-known Geo type/property IDs

The data model is well-structured: block explorers have URL-type properties (website, API URL, docs, GitHub, LinkedIn, X), and networks have **relation** fields pointing to explorer entities — demonstrating cross-entity linking.

Issues Found

Critical

1. SDK version `^0.7.0` — 6+ minor versions behind `package.json:11`

The SDK is at `^0.7.0` while the latest is `0.13.1+`. This is the most outdated version across all reviewed submissions. Breaking changes in 0.8–0.13 include revised `Graph.createEntity` signatures, new IPFS publish flow, and updated ABI contracts.

```
// Current
"@geoprotocol/geo-sdk": "^0.7.0"
// Fix
"@geoprotocol/geo-sdk": "^0.13.1"
```

2. Non-deterministic entity IDs `importer.ts:159-164`

`Graph.createEntity()` generates random UUIDs. Re-running the uploader creates duplicate entities in Geo because each run generates new IDs for the same data.

Per GRC-20 spec, `CreateEntity` is an **upsert** — using `derivedUuidFromString()` would make repeated runs idempotent.

```
// Current — random ID every run
const entity = Graph.createEntity({
  name,
  types: [schema.entityTypeId],
  values: values as any,
  relations,
});

// Fix — deterministic ID from name
import { derivedUuidFromString } from "@geoprotocol/geo-sdk";
const entityId = derivedUuidFromString(name);
const entity = Graph.createEntity({
  id: entityId,
  name,
  types: [schema.entityTypeId],
  values: values as any,
  relations,
});
```

Same issue in `resolveRefEntity()` at `importer.ts:192-196` — relation target entities (e.g., "Etherscan" created as an Explorer when referenced from a Network) also get random IDs.

3. Entity dedup is global, not space-scoped `graph-client.ts:26-42`

`searchEntityByName()` searches all of Geo, not a specific space. If "Ethereum" exists in *any* space (likely), it returns that ID and skips creating the entity in the target space. This means many records may be silently skipped without actually being in the user's space.

```
// Current — matches any entity named "Ethereum" in all of Ge
o
const data = await gql(`{
  search(query: ${JSON.stringify(name)}, first: 10) {
```

```
    id name
  }
}`);

// Should scope to target space:
// search(query: "Ethereum", filter: { spaceId: { is: "..." }
})
```

The `searchEntityByNameAndType()` variant at line 49-72 is better (scopes by type) but still not space-scoped.

4. GraphQL injection in `publishOps` `graph-client.ts:131-138`

`spaceId` is interpolated directly into the query string without using GraphQL variables. While `spaceId` comes from env vars (not user input), this is still a bad pattern and could break on unexpected characters.

```
// Current — string interpolation
const spaceData = await gql(`{
  space(id: "${spaceId}") { ... }
}`);

// Fix — use variables
const spaceData = await gql(
  `query($id: String!) { space(id: $id) { ... } }`,
  { id: spaceId }
);
```

Same issue at lines 159-161 where `address` is interpolated.

5. `toGeoValueType` maps int64/float64 both to `"float"` `importer.ts:18-26`

Both `int64` and `float64` field types are mapped to `"float"` when setting entity values. JavaScript's `number` type can't safely represent all int64 values (precision loss above `Number.MAX_SAFE_INTEGER`). The block-explorers config uses `int64` for the "Launched" year field, which works fine for years, but the generic SDK advertises int64 support that isn't safe.

```
// Current
case "int64": case "float64": return "float";

// Should handle int64 separately or document the limitation
```

Moderate

6. `ROOT_SPACE_ID` is the AI space, not Root `constants.ts:1`

```
export const ROOT_SPACE_ID = "a19c345ab9866679b001d7d2138d88a
1";
```

This is the AI space ID. The actual Geo Root space is `08c4f09378584b7c9b94b82e471d5797` (or similar). The constant is defined but never used in the core logic (configs use `wellKnownPropertyId` directly), so it doesn't cause runtime issues — but it's misleading for anyone extending the code.

7. `as any` casts bypass type safety

Multiple locations:

- `importer.ts:162` — `values as any` when calling `Graph.createEntity`
- `graph-client.ts:126` — `walletClient as any` in `resolvePersonalSpaceId`
- `graph-client.ts:193` — `walletClient as any` in `sendTransaction`

These suggest the SDK types don't align with the code's usage — likely because the SDK is 6+ versions old and the type signatures have diverged.

8. Space ID truncation in `resolvePersonalSpaceId` `graph-client.ts:112`

```
return spaceIdHex.slice(2, 34).toLowerCase();
```

This takes only the first 32 hex characters after `0x` from the contract's `bytes32` return. Geo space IDs are 32 hex chars, but depending on how the contract packs the value (left-aligned vs right-aligned in bytes32), this could truncate the actual ID. Should verify against a known space ID.

9. Silent error swallowing in search functions `graph-client.ts:39-41, 70-71`

Both search functions catch all errors and return `null`. If the Geo API is down or rate-limited, every entity lookup silently "fails" and the importer creates new entities instead of finding existing ones — leading to mass duplication.

```
// Current
} catch {
  return null;  // API error? Network timeout? Treat as "not
found"
}

// Better — log the error, optionally retry
} catch (err) {
  console.warn(`Search failed for "${name}": ${err}`);
  return null;
}
```

10. `networks` field stored as comma-separated text instead of relations `block-explorers-config.json:27-31`

The "Networks" field in block-explorers config is `"type": "text"` containing comma-separated names like `"Ethereum, Gnosis Chain, Base, Optimism"`. Since the Networks bounty creates actual Network entities, these should be relations linking Explorer → Network. The networks config does link Network → Explorer via relations, but the reverse link is a flat string.

Minor

11. No tests

No test files exist anywhere in the repo. The `package.json` has no test script. Given the config-driven architecture, at least schema resolution and the `toGeoValueType` / `toGeoDataType` mappings should have unit tests.

12. `moduleResolution: "bundler"` mismatch `tsconfig.json:5`

The project runs via `tsx` (Node.js ESM) but `tsconfig` uses `moduleResolution: "bundler"`. Should be `"NodeNext"` to match the runtime. This can cause import resolution mismatches (e.g., `.js` extension handling).

13. Relation reference entities lack property values `importer.ts:192-196`

When a relation target (e.g., Explorer entity referenced by a Network) doesn't exist, `resolveRefEntity` creates a bare entity with only a name and type — no description, website, or other properties. If the block-explorers bounty runs first, these entities exist. But if networks runs first, it creates stub explorers that would need manual enrichment.

```
// Current — creates minimal entity
const entity = Graph.createEntity({
  name,
  types: field.relationEntityTypeId ? [field.relationEntityTy
peId] : [],
  // no values — just name + type
});
```

14. File detection heuristic is fragile `upload.ts:28-35`

The uploader distinguishes config from data by checking `Array.isArray(parsed)` vs `parsed.bountyName`. If a bounty folder has multiple JSON objects (e.g., a stale config copy), only the last one wins. No warning is logged.

What's Done Well

- **Config-driven architecture** is significantly better than hardcoded scripts — adding a new bounty requires zero code changes
- **Relation support** between entity types (Network → Explorer) is properly implemented with type-scoped search fallback
- **Well-known IDs** for both entity types and properties — reuses existing Geo schema instead of creating duplicates
- **DAO space support** — `publishOps` handles both personal and DAO spaces with proper member/editor verification
- **Transaction confirmation** — actually waits for receipt unlike other submissions
- **Dry-run mode** — preview ops without publishing

- **Clean separation of concerns** — CLI, importer, graph client, and types are properly modularized
- **README is clear and actionable** — explains setup, usage, and how to add new bounties
- **Data quality** — 65 block explorers and 39 networks with good descriptions, URLs, and cross-references

---

Summary

Severity	Count
Critical	5
Moderate	5
Minor	4

The architecture is the strongest of the bounty submissions reviewed. The config-driven approach is reusable and well-designed. However, the extremely outdated SDK (`^0.7.0`), non-deterministic entity IDs, and global (non-space-scoped) dedup search are serious issues that would cause problems in production — especially duplicate entities on re-runs and incorrect dedup matches from other spaces.

Priority fixes: Update SDK to `^0.13.1`, add `derivedUuidFromString()` for deterministic IDs, scope entity search to target space.